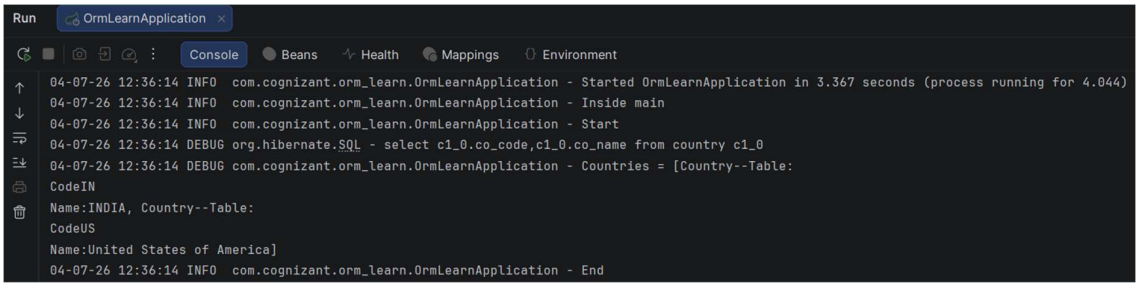
Hands on 1

Spring Data JPA - Quick Example

OrmLearnApplication.java:

```
package com.cognizant.orm_learn;

import com.cognizant.orm_learn.model.Country;
import com.cognizant.orm_learn.service.CountryService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import java.util.List;

@SpringBootApplication
public class OrmLearnApplication {

   private static final Logger LOGGER =
       LoggerFactory.getLogger(OrmLearnApplication.class);
   private static CountryService countryService;

   public static void main(String[] args) {

     ApplicationContext context =
         SpringApplication.run(OrmLearnApplication.class, args);

     countryService = context.getBean(CountryService.class);

     LOGGER.info("Inside main");

     testGetAllCountries();

   }

   private static void testGetAllCountries() {

     LOGGER.info("Start");

     List<Country> countries = countryService.getfullTable();

     LOGGER.debug("Countries = {}", countries);

     LOGGER.info("End");

   }

}
```

Output:



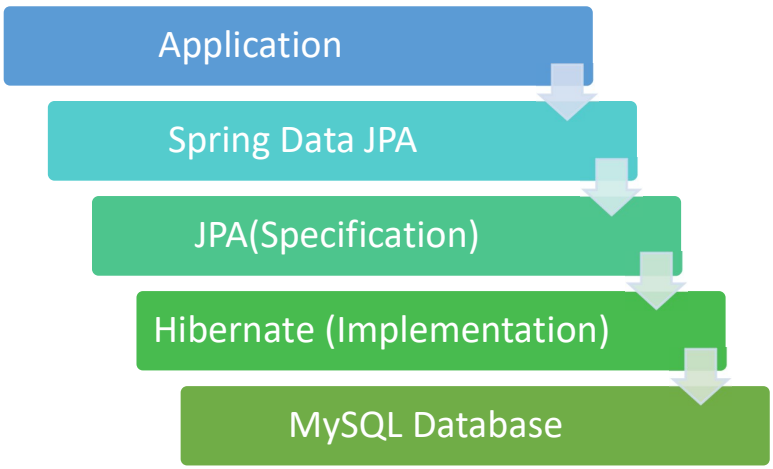
Hands on 2

Difference between JPA, Hibernate and Spring Data JPA

Introduction

Java applications use persistence frameworks to interact with relational databases. **JPA**, **Hibernate**, and **Spring Data JPA** are closely related technologies used for this purpose. JPA defines the standard rules, Hibernate implements those rules, and Spring Data JPA simplifies database operations by reducing boilerplate code.

Relationship Diagram:



Difference Between JPA, Hibernate and Spring Data JPA

Feature	JPA	Hibernate	Spring Data JPA
Definition	Java Persistence API (Specification)	ORM Framework implementing JPA	Spring module built on top of JPA
Type	Specification	Implementation	Abstraction Layer
Purpose	Defines persistence rules	Maps Java objects to database tables	Simplifies database access
Database Operations	Defines APIs only	Performs CRUD operations	Uses Repository interfaces for CRUD
Boilerplate Code	High	Medium	Very Low
Transaction Management	Defines rules	Manual transaction handling	Automatic transaction management using Spring
Example	@Entity, @Id	Session, SessionFactory	JpaRepository, @Repository
SQL Generation	No	Yes	Uses Hibernate internally

Hibernate

```
Session session = factory.openSession();

Transaction tx =
session.beginTransaction();

session.save(employee);

tx.commit();

session.close();
```

Spring Data JPA

```
@Autowired
private EmployeeRepository
employeeRepository;

@Transactional
public void addEmployee(Employee
employee) {

employeeRepository.save(employee);
}
```